```python
import pandas as pd
df=pd.read_csv('/content/credit.csv')
print(df)
```

```
        Time        V1        V2        V3        V4        V5
V6  \
0          0 -1.359807 -0.072781  2.536347  1.378155 -0.338321
0.462388
1          0  1.191857  0.266151  0.166480  0.448154  0.060018 -
0.082361
2          1 -1.358354 -1.340163  1.773209  0.379780 -0.503198
1.800499
3          1 -0.966272 -0.185226  1.792993 -0.863291 -0.010309
1.247203
4          2 -1.158233  0.877737  1.548718  0.403034 -0.407193
0.095921
...      ...       ...       ...       ...       ...       ...       .
..
13949  24754  1.252924 -0.182189 -0.802716 -0.210981  1.916713
3.643624
13950  24756 -0.346979 -2.103284 -0.685061  1.961605 -0.401125
0.473632
13951  24759 -6.053652 -5.988723  0.810413 -0.011811  1.308135 -
0.590803
13952  24759  1.169121 -1.284945  0.032717 -0.681670  0.660598
4.412578
13953  24759 -6.917152  5.854171 -1.652458 -1.488884 -0.833891 -
0.344418

             V7        V8        V9  ...       V21       V22       V23
\
0      0.239599  0.098698  0.363787  ... -0.018307  0.277838 -0.110474

1     -0.078803  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288

2      0.791461  0.247676 -1.514654  ...  0.247998  0.771679  0.909412

3      0.237609  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321

4      0.592941 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458

...         ...       ...       ...  ...       ...       ...       ...

13949 -0.778711  0.818295  1.706962  ... -0.497088 -1.211285  0.043809

13950  1.133816 -0.256528  0.893409  ...  0.359662 -0.316275 -0.864259

13951 -0.725838 -0.234840  1.624646  ... -0.771970  1.474668  3.176363

13952 -1.913115  1.076592  1.501230  ... -0.557596 -0.882435 -0.041523
```

```
13953  0.393789  0.379968  6.133597  ... -1.404681 -1.124694  0.174333

            V24       V25       V26       V27       V28  Amount  Class
0      0.066928  0.128539 -0.189115  0.133558 -0.021053  149.62    0.0
1     -0.339846  0.167170  0.125895 -0.008983  0.014724    2.69    0.0
2     -0.689281 -0.327642 -0.139097 -0.055353 -0.059752  378.66    0.0
3     -1.175575  0.647376 -0.221929  0.062723  0.061458  123.50    0.0
4      0.141267 -0.206010  0.502292  0.219422  0.215153   69.99    0.0
...         ...       ...       ...       ...       ...     ...    ...
13949  0.964159  0.442030  0.261483 -0.051402  0.005112   23.74    0.0
13950 -0.279881  0.491802 -0.353996 -0.149931  0.129795  794.20    0.0
13951 -0.302410  0.052529 -0.373871 -0.700463  2.508443   60.00    0.0
13952  0.975445  0.297229  0.550515  0.015029  0.032067   90.00    0.0
13953 -0.528234  0.990685 -0.035875  1.071374 -0.168831     NaN    NaN

[13954 rows x 31 columns]
```

```
df.isnull().sum()
```

```
Number of non-finite values in the DataFrame:
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
```

```
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    1
Class     1
dtype: int64
```

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.impute import SimpleImputer

# Load the provided DataFrame
# Assuming the DataFrame is named 'data'
# If your DataFrame is named differently, replace 'data' with the
actual name
data = pd.read_csv("/content/credit.csv")  # Replace
"your_file_path_here.csv" with the actual file path

# Handle missing values
imputer = SimpleImputer(strategy='mean')
data_imputed = pd.DataFrame(imputer.fit_transform(data),
columns=data.columns)

# Separate features (X) and target variable (y)
X = data_imputed.drop('Class', axis=1)  # Features
y = data_imputed['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Convert 'Class' column to integer type
data_imputed['Class'] = data_imputed['Class'].astype(int)

# Separate features (X) and target variable (y)
X = data_imputed.drop('Class', axis=1)  # Features
```

```
y = data_imputed['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Initialize Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Predict on the testing set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9987792306440236

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/
_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(
```

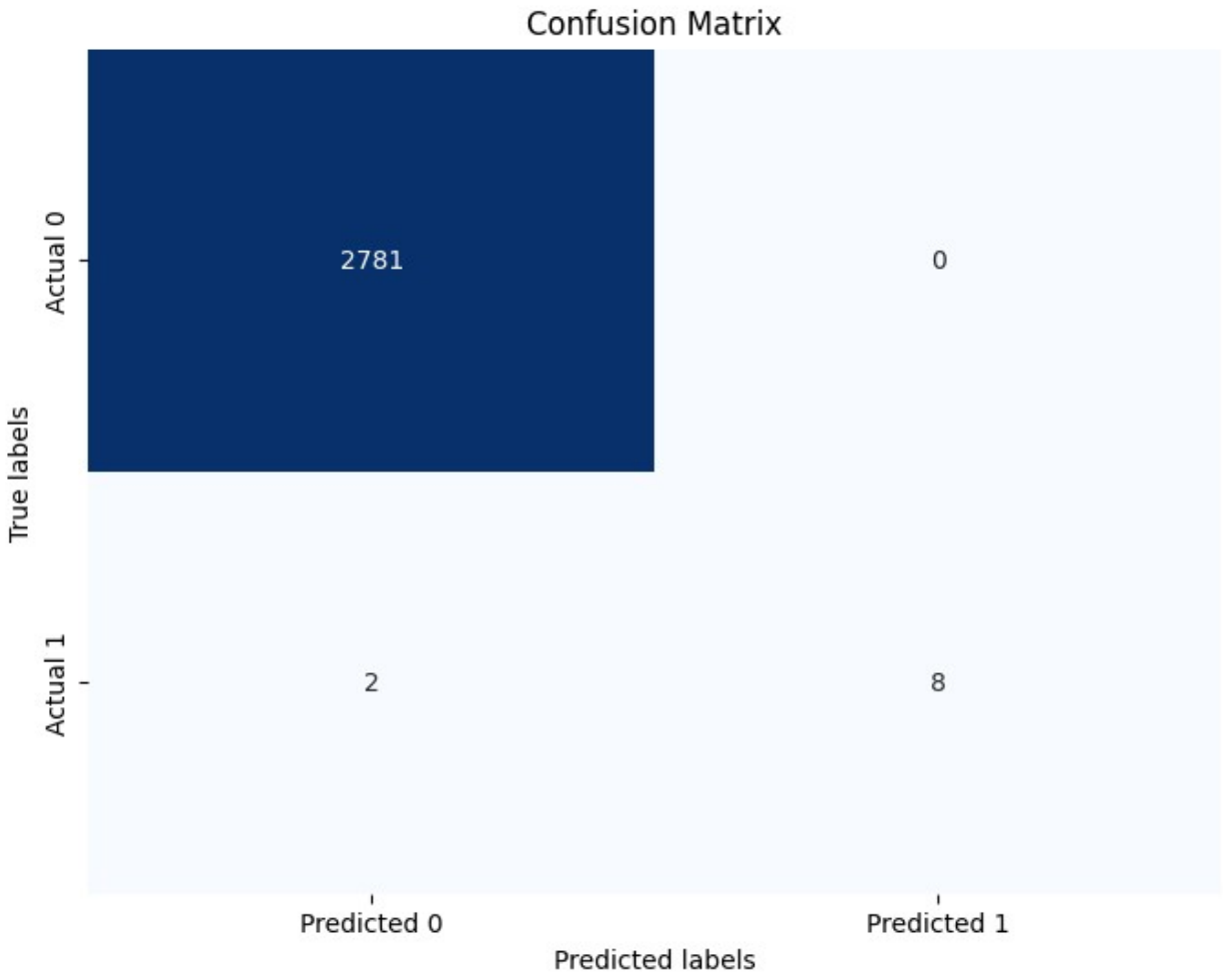
```
#decision tree
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
df['Amount'].fillna(df['Amount'].mean(), inplace=True)
df['Class'].fillna(df['Class'].mode()[0], inplace=True)
df.fillna(df.mean(), inplace=True)
df['Class'] = df['Class'].apply(lambda x: 1 if x > 0 else 0)
X = df.drop(['Class', 'Amount'], axis=1)
y = df['Class']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:",accuracy)
```

```
Accuracy: 0.9992834109638122
```

```
from sklearn.metrics import classification_report


class_report = classification_report(y_test, y_pred)
print("Classification Report:\n", class_report)
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt


conf_matrix = confusion_matrix(y_test, y_pred)

  #confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, cmap="Blues", fmt="d",
cbar=False,
             xticklabels=['Predicted 0', 'Predicted 1'],
                         yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix')
plt.show()
```

```
Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00      2781
           1       1.00      0.80      0.89        10

    accuracy                           1.00      2791
   macro avg       1.00      0.90      0.94      2791
weighted avg       1.00      1.00      1.00      2791
```




```
#logistic c regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score


imputer = SimpleImputer(strategy='mean')
df_imputed = pd.DataFrame(imputer.fit_transform(df),
columns=df.columns)


X = df_imputed.drop('Class', axis=1)
y = df_imputed['Class']


X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
model = LogisticRegression()


model.fit(X_train, y_train)


y_pred = model.predict(X_test)


accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:",accuracy)
```

```
Accuracy: 0.9982085274095306
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/
_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(
```

```
#naive bayes
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt



df_clean = df.dropna()


X = df_clean.drop('Class', axis=1)
y = df_clean['Class']


X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)


nb_classifier = GaussianNB()
```

```
nb_classifier.fit(X_train, y_train)


y_pred = nb_classifier.predict(X_test)


accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)


conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
cbar=False,
                      xticklabels=['Non-Fraud', 'Fraud'],
yticklabels=['Non-Fraud', 'Fraud'])
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix')
plt.show()
```

```
Accuracy: 0.9788606234324615
```


```
#support vector machines


import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Assuming your DataFrame is named 'df'
# Drop rows with missing values
df_cleaned = df.dropna()

# Separate features (X) and target variable (y)
X = df_cleaned.drop('Class', axis=1)  # Features
y = df_cleaned['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

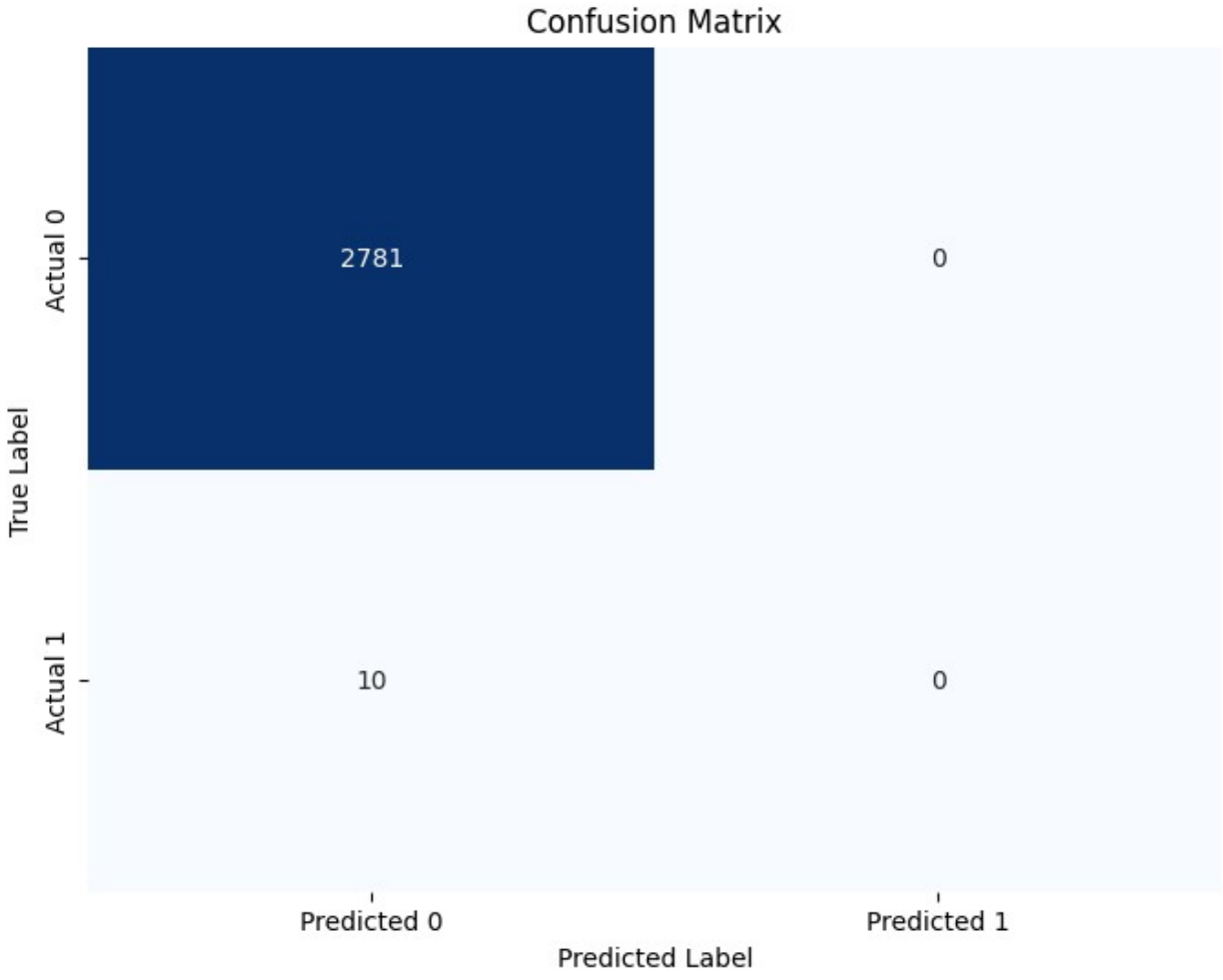
```
# Initialize Support Vector Classifier model
model = SVC()

# Train the model
model.fit(X_train, y_train)

# Predict on the testing set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9964170548190613
```

```
#confusion matrix diagram
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming your DataFrame is named 'df'
# Drop rows with missing values
df_cleaned = df.dropna()

# Separate features (X) and target variable (y)
X = df_cleaned.drop('Class', axis=1)  # Features
y = df_cleaned['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Initialize Support Vector Classifier model
model = SVC()

# Train the model
model.fit(X_train, y_train)

# Predict on the testing set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
```

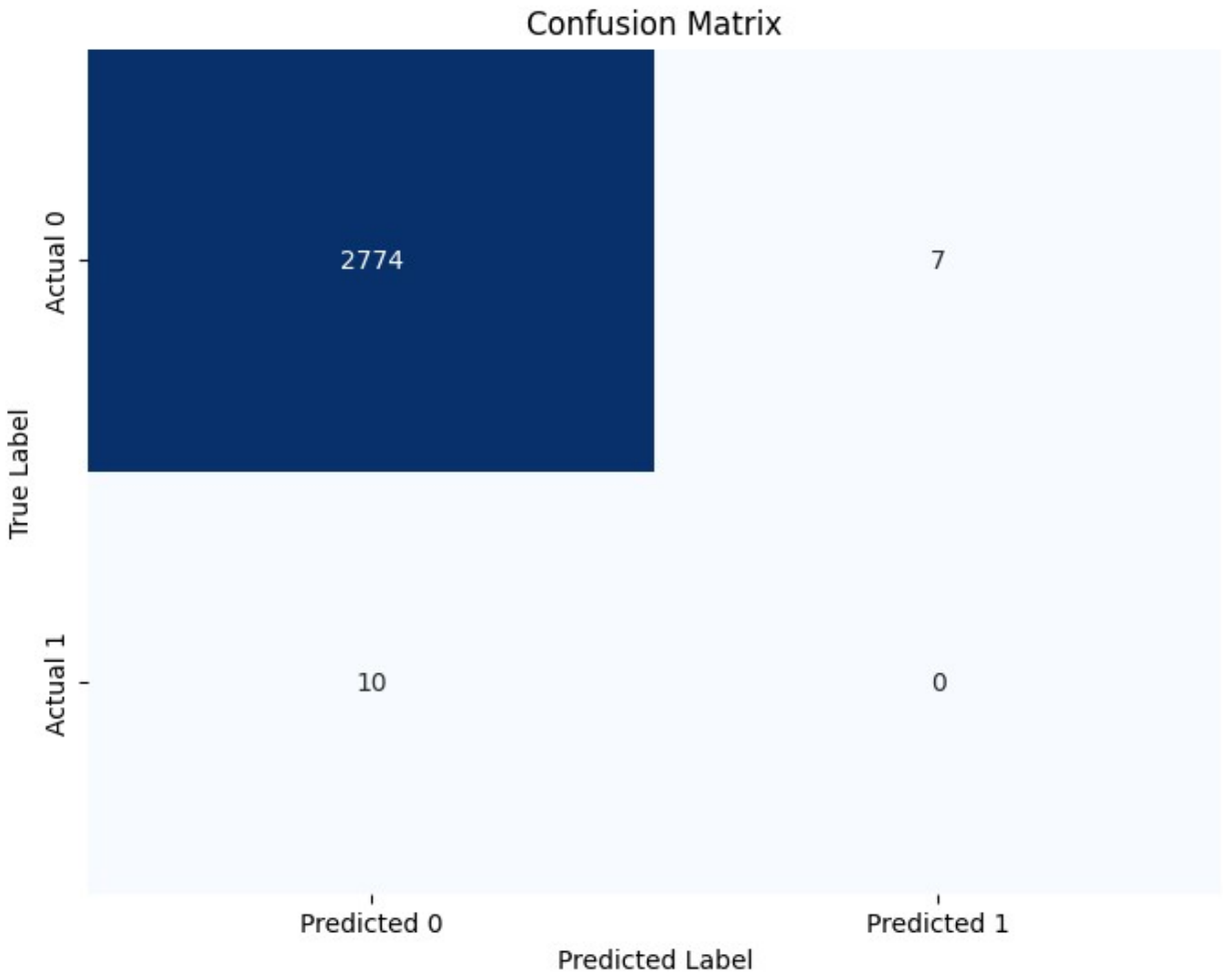
```
# Convert confusion matrix to DataFrame
conf_df = pd.DataFrame(conf_matrix, index=['Actual 0', 'Actual 1'],
columns=['Predicted 0', 'Predicted 1'])

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_df, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Accuracy: 0.9964170548190613
```




```
#multilayer perceptrons
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

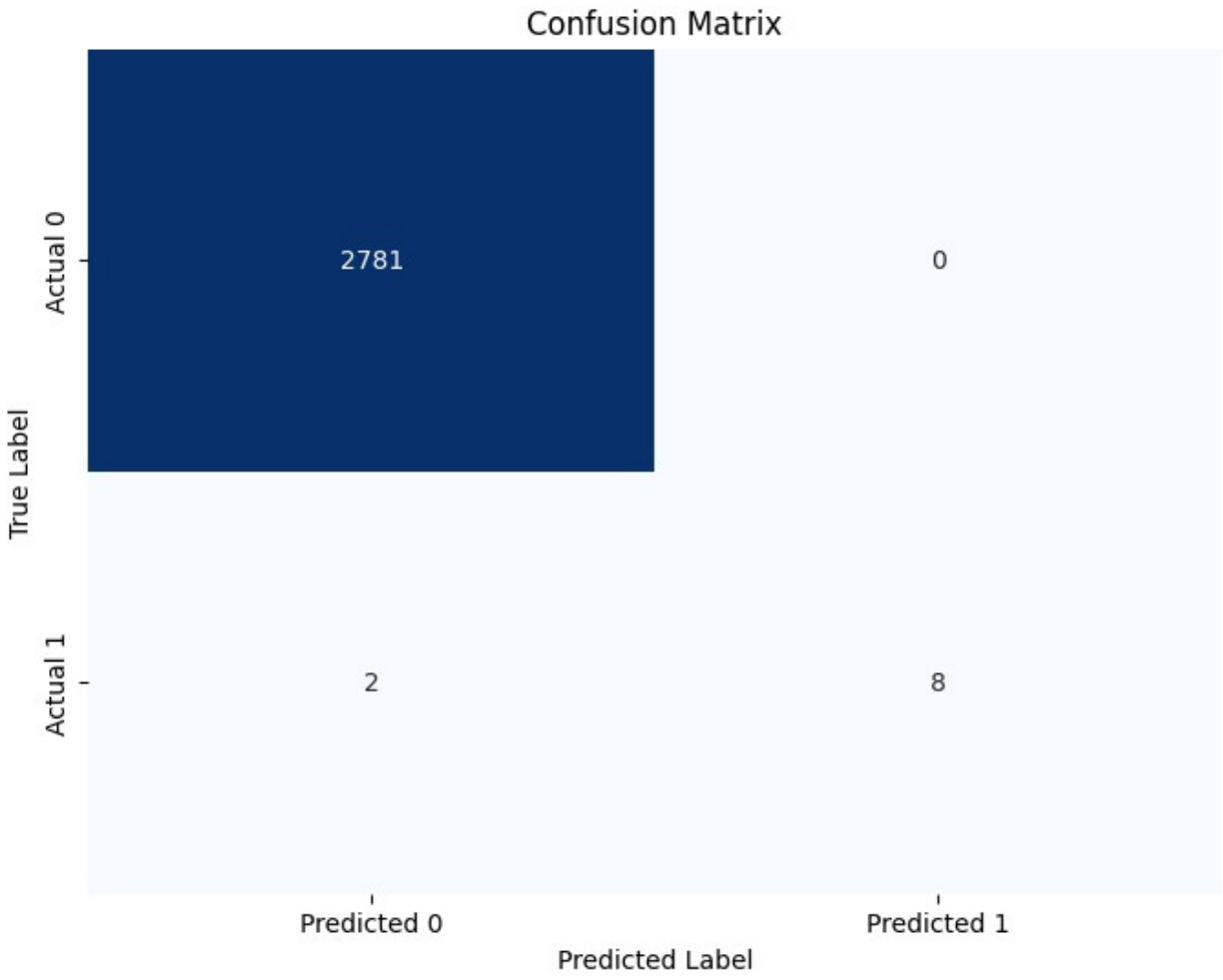
```
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming your DataFrame is named 'df'
# Drop rows with missing values
df_cleaned = df.dropna()

# Separate features (X) and target variable (y)
X = df_cleaned.drop('Class', axis=1)  # Features
y = df_cleaned['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Initialize Multilayer Perceptron (MLP) model
model = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000)

# Train the model
model.fit(X_train, y_train)

# Predict on the testing set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Convert confusion matrix to DataFrame
conf_df = pd.DataFrame(conf_matrix, index=['Actual 0', 'Actual 1'],
columns=['Predicted 0', 'Predicted 1'])

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_df, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

Accuracy: 0.9939089931924041
```




```
#xg boost
import pandas as pd
from sklearn.model_selection import train_test_split
import xgboost as xgb
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Assuming your DataFrame is named 'df'
# Drop rows with missing values
df_cleaned = df.dropna()

# Separate features (X) and target variable (y)
X = df_cleaned.drop('Class', axis=1)  # Features
y = df_cleaned['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
# Initialize XGBoost model
model = xgb.XGBClassifier()

# Train the model
model.fit(X_train, y_train)

# Predict on the testing set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Convert confusion matrix to DataFrame
conf_df = pd.DataFrame(conf_matrix, index=['Actual 0', 'Actual 1'],
columns=['Predicted 0', 'Predicted 1'])

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_df, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

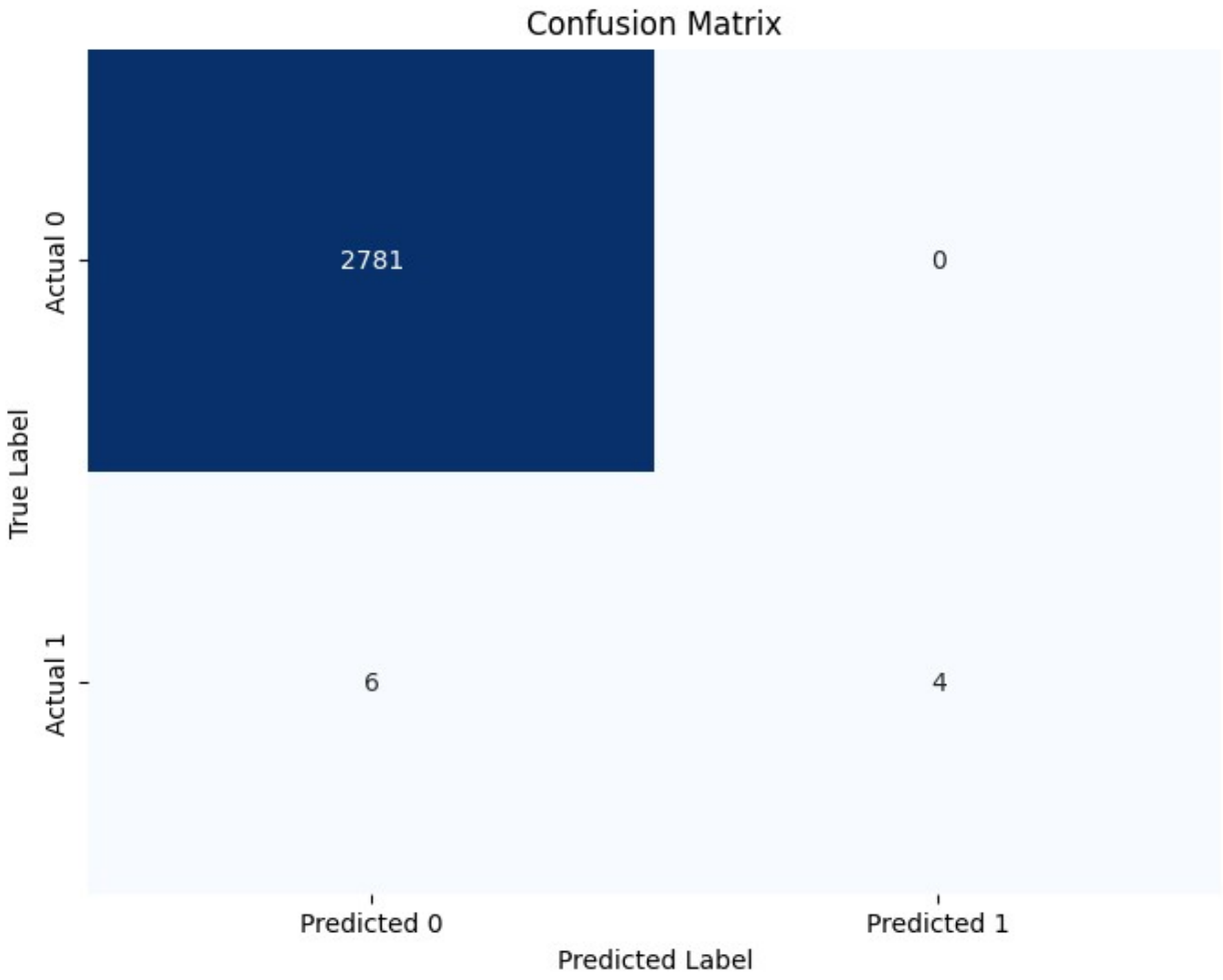
```
Accuracy: 0.9992834109638122
```




```
#convolutional neutral networks
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

# Assuming your DataFrame is named 'df'
# Drop rows with missing values
df_cleaned = df.dropna()

# Separate features (X) and target variable (y)
X = df_cleaned.drop('Class', axis=1)  # Features
y = df_cleaned['Class']  # Target variable

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
# Reshape X data for CNN (assuming it's a 2D image-like data)
X_train = X_train.values.reshape(X_train.shape[0], X_train.shape[1],
1)
X_test = X_test.values.reshape(X_test.shape[0], X_test.shape[1], 1)

# Initialize CNN model
model = models.Sequential([
    layers.Conv1D(32, kernel_size=3, activation='relu',
input_shape=(X_train.shape[1], 1)),
        layers.MaxPooling1D(pool_size=2),
            layers.Flatten(),
                layers.Dense(64, activation='relu'),
                    layers.Dense(1, activation='sigmoid')
                    ])

                    # Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

                    # Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_data=(X_test, y_test), verbose=1)

                    # Predict on the testing set
y_pred = (model.predict(X_test) > 0.5).astype("int32")

                    # Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

                    # Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

                    # Convert confusion matrix to DataFrame
conf_df = pd.DataFrame(conf_matrix, index=['Actual 0', 'Actual 1'],
columns=['Predicted 0', 'Predicted 1'])

                    # Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_df, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

Epoch 1/10
349/349 [==============================] - 3s 4ms/step - loss: 1.8717
- accuracy: 0.9884 - val_loss: 0.5204 - val_accuracy: 0.9914
Epoch 2/10
```

```
349/349 [==============================] - 1s 3ms/step - loss: 0.3614
- accuracy: 0.9916 - val_loss: 0.2372 - val_accuracy: 0.9964
Epoch 3/10
349/349 [==============================] - 1s 4ms/step - loss: 0.1384
- accuracy: 0.9954 - val_loss: 0.1049 - val_accuracy: 0.9979
Epoch 4/10
349/349 [==============================] - 2s 4ms/step - loss: 0.6281
- accuracy: 0.9949 - val_loss: 0.2749 - val_accuracy: 0.9936
Epoch 5/10
349/349 [==============================] - 1s 4ms/step - loss: 0.1524
- accuracy: 0.9961 - val_loss: 0.1860 - val_accuracy: 0.9979
Epoch 6/10
349/349 [==============================] - 1s 3ms/step - loss: 0.1036
- accuracy: 0.9975 - val_loss: 0.4796 - val_accuracy: 0.9964
Epoch 7/10
349/349 [==============================] - 1s 3ms/step - loss: 0.0904
- accuracy: 0.9973 - val_loss: 0.3982 - val_accuracy: 0.9957
Epoch 8/10
349/349 [==============================] - 1s 3ms/step - loss: 0.1489
- accuracy: 0.9969 - val_loss: 0.4151 - val_accuracy: 0.9961
Epoch 9/10
349/349 [==============================] - 1s 3ms/step - loss: 0.1623
- accuracy: 0.9978 - val_loss: 0.3171 - val_accuracy: 0.9968
Epoch 10/10
349/349 [==============================] - 1s 3ms/step - loss: 0.1220
- accuracy: 0.9979 - val_loss: 0.2012 - val_accuracy: 0.9979
88/88 [==============================] - 0s 2ms/step
Accuracy: 0.9978502328914367
```


Confusion Matrix
Actual 0
Actual 1
True Label
2781
0
6
4
Predicted 0
Predicted 1
Predicted Label